ПРИЛОЖЕНИЕ К ПРОЕКТУ

Безопасный прием чаевых по QR-коду: комиссии, проверка платежей и защита от повторного списания.

СОСТАВ КОМАНДЫ AI UNITED

Участник	Роль
Белый Андрей Сергеевич	Презентатор
Передеряев Иван Иванович	Разработчик
Барсуков Никита Денисович	Аналитик
Василянский Артем Сергеевич	Архитектор

Ключевые результаты

- Прототип на C++17: SQLite, OpenSSL, консольный запуск и эмулятор платежного шлюза.
- 12 автоматизированных проверок завершились успешно: 5 обязательных сценариев и 7 дополнительных.
- Оплата в целых копейках, комиссия 5%, распределение пула по часам без потери остатка.
- Подписанный QR, контроль ролей, идемпотентность, аудит и сверка неопределенных статусов.

Статус: учебный демонстрационный прототип. Реальные платежи, вывод средств и банковские интеграции не выполняются.

02 Постановка задачи и требования

Проблема: наличные используются все реже, а чаевые требуют удобного, предсказуемого и безопасного канала оплаты.

Функциональные требования (FR)

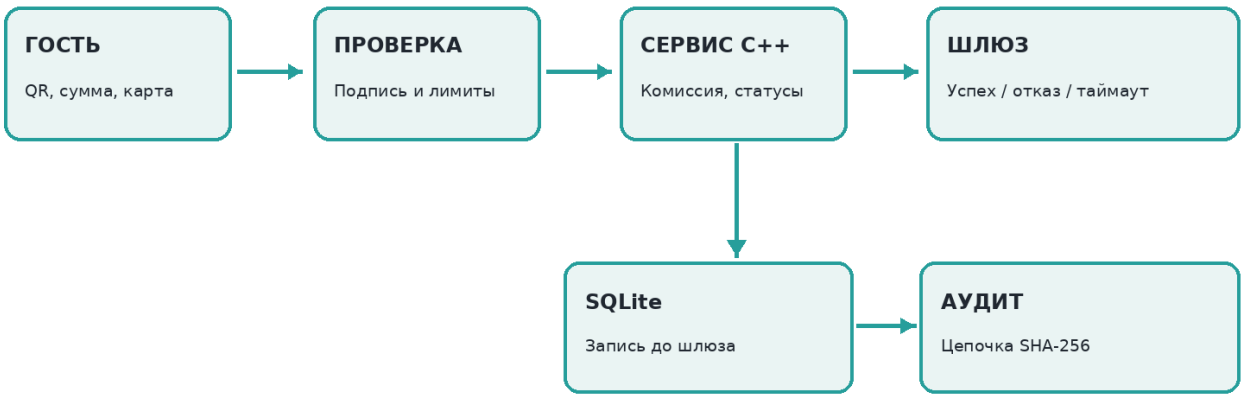
Код	Требование	Кто выполняет
FR-1	Выпуск QR для привязки к заведению и сотруднику	Менеджер / админ
FR-2	Проверка криптографической подписи и срока QR	Сервис
FR-3	Оплата чаевых от 50 до 10 000 руб.	Гость
FR-4	Возможность оплатить комиссию за счет гостя	Гость
FR-5	Расчет комиссии 5% и суммы официанту	Сервис
FR-6	Распределение пула пропорционально часам	Менеджер
FR-7	Просмотр собственных успешных чаевых	Официант
FR-8	Сверка зависших операций со шлюзом	Сервис
FR-9	Контроль целостности журнала операций	Аудитор

Нефункциональные требования (NFR)

Код	Требование / критерий
NFR-1	Повтор запроса не приводит ко второму обращению к шлюзу.
NFR-2	Неопределенный результат не считается автоматическим отказом; выполняется сверка.
NFR-3	Полный PAN и телефон не сохраняются в SQLite.
NFR-4	Финансовые суммы хранятся в целых копейках.
NFR-5	Изменение записи в журнале выявляется проверкой цепочки.
NFR-6	Таймаут 3 с и p95 менее 500 мс - проектные цели; нагрузочных измерений нет.

03 Архитектура решения

Поток данных от ввода гостя до сохранения итога в SQLite и записи в журнале.



Назначение компонентов

- Гость вводит сумму и тестовую карту; менеджер ранее выпускает QR с HMAC-SHA256.
- Проверка подтверждает подлинность QR, права роли, сумму, формат Idempotency-Key и алгоритм Луна.
- Сервис рассчитывает комиссию и сохраняет запись CREATED, затем PENDING до вызова эмулятора.
- Шлюз моделирует успех, отказ и ситуацию «деньги списаны, ответ потерян».
- SQLite хранит статус и маску PAN, аудит связывает записи хешами предыдущих событий.

Границы реализации

Сервис реализован как запускаемый из консоли учебный прототип. Маршрут POST /tips показан как целевой контракт будущего HTTP API, но отдельный HTTP-сервер и публичная страница QR не реализованы. tips.example - демонстрационный адрес, он не принимает оплату.

04 Словарь данных и валидация

Входные данные проверяются до обращения к эмулятору банка.

Поле	Тип / формат	Правило	Ошибка
qr_url	string	Подписанный QR нужного заведения; не истек срок	QR_MALFORMED / QR_BAD_SIGNATURE / QR_EXPIRED
amount_kop	int64	От 5000 до 1000000 коп. включительно	AMOUNT_OUT_OF_RANGE
pan	string	13-19 цифр и контроль Луна	CARD_INVALID
idempotency_key	UUID string	UUID в канонической записи	IDEM_KEY_INVALID
cover_fee	bool	Оплачивает ли гость комиссию	-
comment	UTF-8 string	До 800 байт в текущем прототипе	COMMENT_TOO_LONG
employee_id	string	Присутствует в действующей записи заведения	EMPLOYEE_NOT_FOUND

Пример запроса на уровне модели C++

- qr_url = подписанная ссылка на сотрудника emp-1;
- amount_kop = 30000, pan = тестовая карта 4242424242424242;
- idempotency_key = UUID, cover_fee = false, comment = "Спасибо".

Важное разграничение

PAN используется только как тестовый ввод эмулятора и для вычисления отпечатка запроса. В SQLite сохраняются только маска карты и криптографический отпечаток. В промышленной версии карточные данные должны поступать напрямую на защищенную платежную форму банка, сервис получает токен, а не PAN.

Коды ошибок

Проверка подписанного QR: 403; истекший QR: 410; конфликт Idempotency-Key: 409; нет сотрудника: 404. Проверки ролей возвращают FORBIDDEN (403). В консольной модели коды существуют как свойства TipError и имитируют будущие HTTP-ответы.

05 Алгоритм платежа и модель статусов

1. Проверить разрешение роли GUEST и корректность Idempotency-Key.

2. Если этот ключ уже использовался: сравнить отпечаток запроса и вернуть предыдущую запись, не списывая повторно.

3. Проверить QR, сумму, карту, комментарий, активность сотрудника.

4. Рассчитать charge, fee, net; создать запись CREATED, перевести в PENDING.

5. Вызвать эмулятор шлюза и присвоить SUCCEEDED, DECLINED или PENDING_UNKNOWN.

6. При PENDING_UNKNOWN запросить факт списания у шлюза через reconcile().

Разрешенные переходы состояний

Текущий статус	Разрешенные следующие статусы
CREATED	PENDING
PENDING	SUCCEEDED, DECLINED, PENDING_UNKNOWN
PENDING_UNKNOWN	SUCCEEDED, DECLINED
SUCCEEDED	REFUNDED (модель переходов, отдельная операция возврата не реализована)
DECLINED / REFUNDED	Новых переходов нет

Почему требуется идемпотентность

При повторном клике клиент повторно использует тот же UUID. Уникальное ограничение в SQLite связывает его с единственной операцией. С тем же телом возвращается сохраненный платеж (replayed=true); с другой суммой - IDEM_CONFLICT. Для промышленной многопоточной версии нужны транзакции и работа с конкурентными запросами, что не проверялось.

06 Комиссия, общий пул и распределение чаевых

Пример на сумме 300 руб.

Режим	Гость платит	Комиссия	Официанту
Комиссия вычитается	300,00 руб.	15,00 руб.	285,00 руб.
Гость покрывает комиссию	315,00 руб.	15,00 руб.	300,00 руб.

Формула: fee_kop = (amount_kop * 5) / 100; целочисленное деление отбрасывает дробную часть копейки. Все суммы представлены int64_t, для комиссии не используется float.

Оплата в общий пул и распределение

Сотрудник	Часы	Доля
Анна	8	444,45 руб.
Иван	6	333,33 руб.
Олег	4	222,22 руб.
Итого	18	1 000,00 руб.

Гость может отправить чаевые не только конкретному официанту, но и в общий пул смены. В этом случае поле to_pool=true переводит net_kop на баланс заведения, а менеджер затем распределяет накопленную сумму по часам через distribute_pool(). Ниже показан пример уже распределенного пула 1 000 руб. для смен 8 / 6 / 4. Сначала каждая доля вычисляется целыми копейками пропорционально часам. Нераспределенные копейки назначаются начиная с самых длинных смен, при равных часах используется порядок идентификатора. Тест для суммы 1 000,01 руб. подтверждает сохранение суммы долей.

Точность и ограничение

В расчетах используется целочисленная арифметика; промежуточное произведение при делении пула хранится в расширенном целочисленном типе компилятора GCC/Clang. На MSVC этот участок требует адаптации. Доли времени в текущем прототипе целочисленные (например, 8 часов), дробные смены не поддерживаются.

07 Безопасность и контроль доступа

Уровни защиты

- HMAC-SHA256: проверка подписанного QR с постоянным по времени сравнением подписи.
- SHA-256 + соль: телефон не хранится в открытом виде.
- Маскирование карты: в базе находится, например, 4242 **** **** 4242.
- SQLite: отдельные таблицы employee, payment, audit; статусный автомат не допускает неверных переходов.
- Аудит: связанная хеш-цепочка позволяет обнаружить изменение ранее записанного события.

Матрица ролей (проектные разрешения)

Роль	Что разрешено
GUEST	Создать оплату чаевых
WAITER	Просматривать свои успешные чаевые
MANAGER	Выпускать QR, смотреть данные заведения, распределять пул - последние две функции заданы в правах, но не вынесены в публичные команды
ADMIN	Выпускать QR; управление заведениями и ключами - запланированные функции
AUDITOR	Читать журнал - роль описана, отдельной команды чтения пока нет

Честные ограничения

Ключ QR и соль в демо имеют учебные значения. Их нельзя применять в реальном сервисе. Ротация ключей, управление доступом отдельных аккаунтов, проверка QR по фотографии официанта и защищенная инфраструктура ключей остаются планом развития.

08 Обработка сбоев и сверка

Ситуация	Реакция сервиса	Что сообщаем гостю
Банк подтвердил оплату	SUCCEEDED	Платеж принят
Банк отказал	DECLINED; чаевые не зачисляются	Оплата отклонена
Шлюз не ответил	PENDING_UNKNOWN	Проверяем платеж; не платите повторно
Повтор с тем же ключом	Возвращается запись без нового списания	Тот же результат
Испорчен QR	QR_BAD_SIGNATURE / QR_MALFORMED	QR недействителен
Изменен старый аудит	verify_audit() = false	Сигнал для аудитора

Сверка неопределенного платежа

Карта ...0119 моделирует ситуацию, в которой эмулятор провел списание, но ответ не дошел до сервиса. Первая операция получает PENDING_UNKNOWN. При вызове reconcile() сервис проверяет запись шлюза и переводит ее в SUCCEEDED. Повторную оплату гостю в этот момент предлагать нельзя.

Почему запись создается до шлюза

Если шлюз успел списать деньги, а приложение не получило ответ, запись операции все равно остается в SQLite. Ее можно найти и сверить. Важно: в учебной модели нет фонового планировщика - reconcile() вызывается вручную в демо; трехсекундная задержка не симулируется реальным ожиданием.

09 Техническая реализация на C++17

Файлы репозитория

Файл	Назначение
cpp/tips_service.hpp	Типы данных, интерфейс TipService
cpp/tips_service.cpp	QR, HMAC-SHA256, SQLite, комиссия, аудит, проверка статусов
cpp/demo.cpp	Шесть сценариев, демонстрация результатов
cpp/tests.cpp	12 автоматизированных проверок, включая оплату в общий пул
CMakeLists.txt	Сборка tips_service, tips_demo, tips_tests
README.md	Описание проекта и инструкция по сборке

Исходный код проекта:https://github.com/devIreum/ai-united-online-tips

Требования к среде

- Компилятор C++17 (в текущей сборке - g++), CMake 3.16 или новее.
- Библиотеки разработки SQLite3 и OpenSSL Crypto.
- Интернет и реальный банковский API для демонстрации не нужны.

Сборка и запуск

```
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure
./build/tips_demo
```

Интерпретация

Прототип демонстрирует внутреннюю бизнес-логику, но не является платежным сервером. Программа использует только тестовые номера карт со специальным поведением в эмуляторе.

10 Пять обязательных тест-кейсов

Факт зафиксирован реальным запуском C++ тестов 07.11.2026 в подготовленной среде.

№	Сценарий	Вход и ожидание	Факт
1	Успех	300 руб., карта ...4242; SUCCEEDED, 285 руб. официанту	PASS: SUCCEEDED, fee 15 руб., net 285 руб.
2	Отказ банка	Карта ...0002; DECLINED, нет начисления	PASS: DECLINED, список начислений пуст
3	Неверные данные	10 руб.; неверный PAN, QR и UUID	PASS: 4 ожидаемых ошибки, без вызова шлюза
4	Повтор	Два запроса с одним UUID; 1 обращение в шлюз	PASS: один id, replayed=true, IDEM_CONFLICT при другой сумме
5	Таймаут	Карта ...0119; PENDING_UNKNOWN, затем сверка	PASS: PENDING_UNKNOWN -> SUCCEEDED

Итог запуска

12 PASS / 0 FAIL. В ctest зарегистрирован один тестовый исполняемый файл tips_tests, внутри которого запускаются 12 самостоятельных проверок. Поэтому ctest показывает «1/1 test», а консоль tips_tests показывает «12 passed, 0 failed». Это разные уровни подсчета, противоречия нет.

Пример кода результата

[PASS] 1. Successful payment

[PASS] 5. Timeout and reconciliation

[PASS] 12. Payment can go to the common pool and be distributed

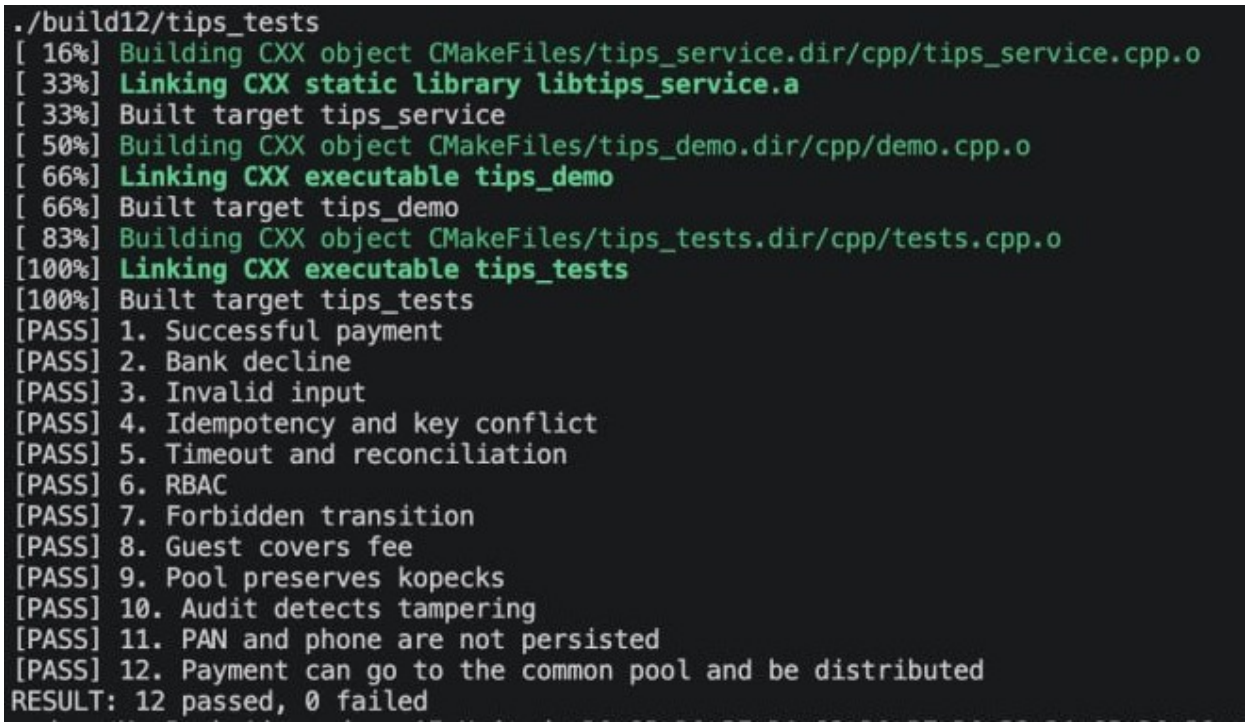
RESULT: 12 passed, 0 failed

11 Дополнительные проверки и результаты

Добавлен связанный сценарий: гость оплачивает чаевые в общий пул, после чего менеджер распределяет сумму по сотрудникам.

№	Тест	Факт
6	Доступ по ролям	FORBIDDEN для официанта при попытке выпуска QR
7	Запрещенный переход	BAD_TRANSITION для DECLINED -> SUCCEEDED
8	Комиссия за счет гостя	315 / 15 / 300 руб.
9	Пул 1 000,01 руб.	Сумма долей в копейках совпадает с пулом
10	Нарушение аудита	После изменения записи verify_audit() возвращает false
11	Защита приватных данных	Полный PAN и телефон отсутствуют в сериализованной SQLite
12	Оплата в общий пул	Платеж с to_pool=true попадает в общий пул и затем распределяется менеджером

Фрагмент консольного запуска



Настоящий скриншот терминала VS Code на MacBook: сборка C++ и 12 успешно пройденных тестов. Текстовые результаты также сохранены в materials/test_output_cpp.txt и materials/demo_output_cpp.txt; исходный код находится в cpp/.

12 Риски, ограничения и план развития

Риск	Примененная мера / перспектива
Подделка подписанного QR	HMAC-SHA256. Подмену на настоящий QR другой точки одной подписью не выявить.
Два списания	Idempotency-Key, UNIQUE в SQLite; конкурентные запросы требуют отдельного исследования.
Неизвестный результат банка	PENDING_UNKNOWN, сверка и запрет повторной оплаты до выяснения.
Утечка QR-ключа	Для продакшена: отдельное хранилище секретов и ротация.
Утечка базы	Нет полного PAN и телефона, есть хеш и маска. Это снижение ущерба, не полная защита.
Рост нагрузки	Для продакшена: PostgreSQL, очередь событий, фоновые воркеры, мониторинг.

Что не реализовано в прототипе

- Настоящий эквайринг, токенизация карт платежным провайдером, выплаты сотрудникам.
- HTTP API, веб-страница оплаты, авторизация реальных аккаунтов, QR для фактического сканирования.
- Хранилище постоянных платежей, фоновые задания, резервное копирование, ротация ключей.
- Нагрузочные испытания, полноценная оценка угроз, PCI DSS и юридическая подготовка.

Возможности развития

Сначала прототип API и интерфейса с идентификаторами заведений и сотрудников; далее интеграция с песочницей эквайринга, тестовые выплаты, многопоточные тесты идемпотентности и мониторинг. После этого можно обсуждать промышленную эксплуатацию.

13 Журнал промптов и участия ИИ

Ключевые обращения, сгруппированные по задачам. Формулировки ранних запросов восстановлены по исходному приложению, а не выданы за дословную переписку.

Дата	Цель и запрос	Результат	Как проверили
06.11.2026	Разбор кейса. «Помоги выделить требования и этапы работы по инструкции чемпионата».	Черновик FR/NFR, список функций и границ прототипа.	Требования сопоставлены с условиями кейса и памяткой чемпионата.
06.11.2026	Проектирование логики. «Продумай подписанные QR, статусы, роли и защиту от повтора».	Схема архитектуры, модель статусов, подход к аудиту и расчету комиссии.	Схема и модель состояний сопоставлены с реализованными сценариями.
06.11.2026	Тестирование. «Подготовь тесты успеха, отказа, ошибок, повтора и таймаута».	Набор основных и дополнительных проверок.	12 PASS / 0 FAIL, фактический запуск tips_tests на MacBook 07.11.2026.
06-07.11.2026	Подготовка к защите. «Составь сложные вопросы жюри о QR, рисках, комиссиях и отказах».	Черновик коротких объяснений и честных ограничений.	Список тем соотнесен с архитектурой, RBAC, расчетами и обработкой сбоев.
07.11.2026	Оформление. «Подготовь презентацию по шаблону, приложение, документацию и журнал ИИ».	Слайды на 8 страниц, техническое приложение, инструкции по сборке.	Проверены структура и объем документов, результаты тестов и состав команды.

14 Матрица вклада команды

Участник	Роль	Зона ответственности в проекте	Доля
Белый Андрей Сергеевич	Презентатор	Презентация, оформление итоговых материалов, представление результатов.	25%
Передеряев Иван Иванович	Разработчик	Код C++17, тесты, сборка и демонстрация работы сервиса.	25%
Барсуков Никита Денисович	Аналитик	Функциональные требования, расчеты комиссии, сценарии тестирования.	25%
Василянский Артем Сергеевич	Архитектор	Архитектура, модель статусов, безопасность, RBAC и аудит.	25%